

Break the Login – Atacarea și securizarea autentificării

Proiect Dezvoltarea Aplicațiilor Software Securizate - Raport Scris

Șerba Ioana-Raluca | grupa 341 | an universitar 2025-2026

Link Prezentare Video: <https://1drv.ms/v/c/fb23350a459c73f5/IQDXuyDEPugSRJP9kelPP5PuAQd316Y9IMJnYyAVxsS5qBE?e=CtTowA>

Link Repo Github: https://github.com/innofuxkingfun/DASS_BreakTheLogin

I. Introducere

Aplicația prezentată, conform numelui proiectului “Break The Login”, este o aplicație relativ simplă care se focusează pe funcționalitățile deregistrare și autentificare, cu o parte minimă de CRUD și search. În cadrul aplicației, utilizatorii își pot crea conturi cu email, parolă și rol, și ulterior, după crearea cu succes a contului, se pot loga. Odată ce contul există, se poate solicita și un process de schimbare a parolei prin primirea unui token (care, în scenariul ideal ar fi fost trimis pe mail, însă pentru demo va fi doar un token afișat). Odată logați, utilizatorii pot crea tickete noi și își pot vedea propriile tickete dacă au rolul de ‘Analyst’, sau dacă au rolul de ‘Manager’ pot vedea toate ticketele din aplicație, pot edita statusul acestora și pot vizualiza audit-logurile.

Arhitectură proiectului este o simplă arhitectură clasică de Flask: app.py este aplicația principală și conține toate funcționalitățile serverului, configurațiile și rutarea paginilor. Database.py conține funcționalitățile legate de baza de date, folosind codul sql din schemă.sql pentru structura acesteia, baza de date în sine fiind ținută în database.db. Iar, în final, paginile html sunt toate ținute în folderul templates.

II. Setup mediu

Pentru mașina virtuală, am folosit Oracle Virtual Box 7.2.8 r173730, cu Windows 10(64-bit), pe un host de Windows 10. Pentru o conexiune funcțională la internet și, cel mai important, la serverul local, am realizat o conexiune de tip Bridged Adapter (Realtek Gaming GbE Family Controller).

Că și baza de date, am folosit SQLite3, întrucât e ușor de folosit și rulat local, și s-a dovedit a fi îndeajuns pentru acest proiect. Ca și framework am folosit Python Flask, însă, cel puțin rulat din PyCharm, acesta rulează strict local. Pentru a putea fi accesat din VM, în ‘__main__’, am adăugat parametrii host='0.0.0.0', port=5000 în funcția run, pentru a facilita acceptarea cererilor externe de conexiune indiferent de adresa ip sau adaptorul de rețea. În continuare, rularea normală poate să sară acest pas, așa că am rulat direct din terminal folosind comanda

‘python app.py’. În așa fel, am reușit să access aplicația din VM la adresa <http://192.168.1.175:5000>.

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.175:5000
```

III. Implementare MVP

Pentru prima variantă, vulnerabilă, a proiectului, am implementat următoarele aspecte cum urmează:

Auth – La nivel de bază de date, am folosit structura recomandată în documentația proiectului. Primul pas din autentificare este, desigur, înregistrarea, care se realizează printr-un formular simplu, cu email, parolă și un dropdown pentru alegerea rolului. La finalizarea completării formularului, se trimite un request de tip POST către adresa /register. Funcția ia informațiile fără să se le verifice, și le pune în baza de date exact așa cum sunt primite (adică, parolă în clar). În această versiune de proiect, cea mai probabilă eroare este posibilă datorită constrăințului de tip UNIQUE pe coloana email, care verifică să nu se repete datele din acea coloană însă în rest, atât timp cât funcția de INSERT se realizează cu succes, s-a creat un utilizator nou.

Pentru login, este un process foarte asemănător cu un formular și un request de tip POST către /login, care verifică dacă există acel utilizator cu acel email, și respectiv dacă și parolă este corectă, afișând mesaje de eroare pentru fiecare caz. Nu există limita de încercări, odată ce este corect, se crează un cookie care reține email-ul utilizatorului în plain-text și care atestă că acesta este logat.

Adițional, mai este funcționalitatea de a reseta parolă: prima oară se introduce o adresa de email, pe care utilizatorul presupune că are cont. Se afișează un token (pentru această versiune, un token static, reutilizabil) (ruta /forgot-password-email; la POST, se trimite adresa de email și se redirecționează către /forgot-password-token-verification), iar dacă se introduce token-ul corect, se trece la pasul final de schimbare a parolei (/forgot-password-reset). În acest punct, emailul a fost transmis de la prima ruta și nu trebuie reintrodus, trebuie doar introdusă parolă nouă. Aplicația verifică dacă email-ul este unul valid, care are cont, însă în rest doar reține parolă în plain-text.

CRUD – Partea această am păstrat-o simplă și este majoritar în jurul ticketelor. Orice utilizator logat poate crea un ticket nou prin completarea unui formular simplu, care face un request POST către /tickets. Cei cu rolul ‘Analyst’ își pot vedea propriile tickete, însă

doar utilizatorii cu rol 'Manager' pot vedea toate ticketele din baza de date (cerere GET către /tickets) și modifică statusul acestora (request POST către /tickets/<ticket_id>), pe lângă vizualizarea log-urilor de audit (GET către /audit-logs).

Search – Această funcționalitate se află tot pe partea de tickete, orice utilizator având dreptul să caute în lista ticketelor la care are drepturi de acces după titlu, descriere, severitate sau status. Pentru această versiune a proiectului, acesta funcție de realizează prin concatenarea inputului direct în string-ul de query și este un request de tip POST către /tickets. În varianta securizată a aplicației, am folosit query-uri parametrizate pentru a evita atacuri de tip SQL-Injection.

IV. Prezentare vulnerabilități

Vulnerabilitățile sunt cum urmează:

1. **Password Policy slab** – Aplicația permite setarea și utilizarea unor parole foarte slabe, neavând nici-un fel de verificare asupra inputului oferit că parolă atâta timp cât nu este nulă. Utilizatorii nu sunt deloc obligați să folosească un amestec de caractere (uppercase, lowercase, numerele, caractere speciale, etc.) sau ca aceasta să aibă o lungime minimă, iar astfel nu se impune un standard minim de complexitate a parolei. Prin această vulnerabilitate, expunem utilizatorii la un atac de tip brute force, prin care atacatorului îi este mult mai ușor să încerce să ghicească parolă (precum, în continuare, cea mai comună parolă este 12345 sau password), sau atac de tip credential stuffing, în care atacatorul utilizează credentiale furate de la alte site-uri și le utilizează la încercarea să între în cont-ul aplicației care are credentiale asemănătoare. (OWASP – Testing for Weak Password Policy [WSTG-ATHN-07]: https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/04-Authentication_Testing/07-Testing_for_Weak_Password_Policy)
2. **Stocare nesigură a parolelor** – Aplicația reține parolele în clar, iar orice atacator care face rost la acces la baza de date, și poate să extragă tabelul de utilizatori, are access automat și direct la parolele tuturor conturilor, fără că acesta să aibă nevoie de resurse în plus să încerce să „spargă” parolele. (OWASP - Password Plaintext Storage: https://owasp.org/www-community/vulnerabilities/Password_Plaintext_Storage)
3. **Brute force / lipsă rate limiting** – Aplicația nu ține cont de câte încercări de login s-au efectuat asupra unui cont, astfel un atacator de tip brute force nu este deloc descurajat sau blocat din a încerca un număr infinit de parole. (OWASP - API2:2023 Broken

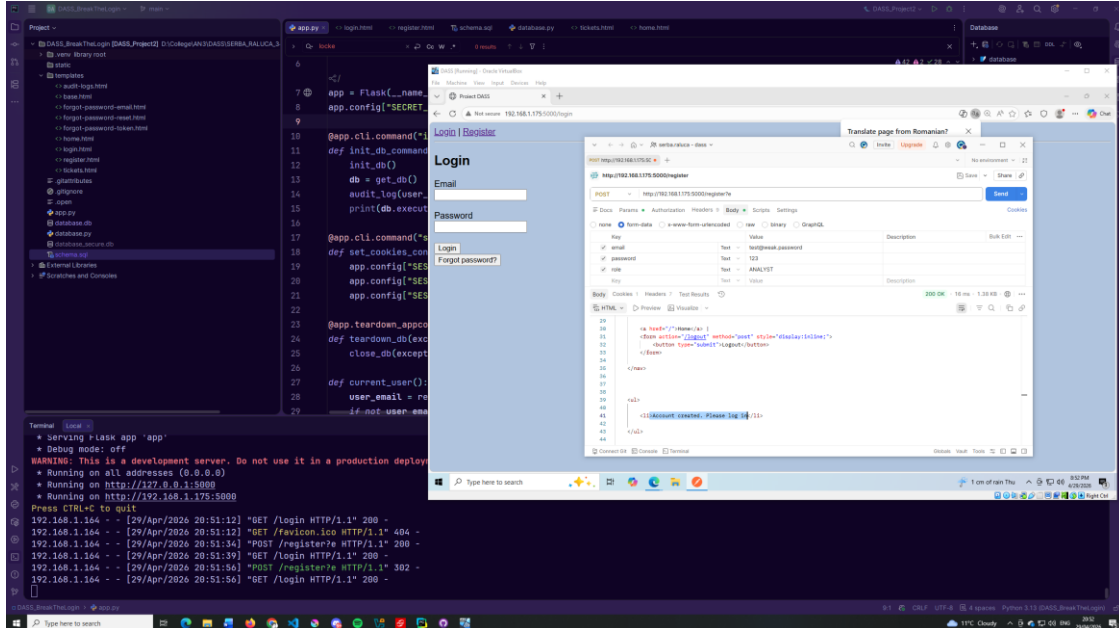
Authentication: <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>)

4. **User Enumeration** - O vulnerabilitate în plus care poate ajuta un atacator de tip brute force, la fel că la punctul anterior, este specificarea dacă din credentialele introduse, care dintre email sau parolă sunt greșite. (OWASP - Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html)
5. **Gestionare nesigură a sesiunilor** – Aplicația are configurate cookie-urile, în mod explicit, să aibă flag-urile de HttpOnly, SameSite și Secure setate că false, astfel făcând aplicația vulnerabilă la scripturi (JavaScript, VBScript etc.) care accesează cookie-urile, la trimiterea și primirea de cookie-uri de pe alt domain și la utilizarea unor conexiuni nesecurizate. Aceste vulnerabilități oferă unui atacator multe cai de acces la a fură datele unui cookie, a le trimite către alte site-uri sau a trimite cookie-uri nesigure către aplicația noastră. (OWST – Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html)
6. **Resetare parolă nesigură** – Pentru resetarea parolei, token-ul utilizat este unul incredibil de predictibil, și reutilizabil, care nu expiră (123). Prin această vulnerabilitate, atacatorul poate reseta parolă unui cont la care nu ar avea drepturi în mod normal, doar prin ghicirea tokenului și introducerea unei adrese de email care are deja cont. (OWASP - API2:2023 Broken Authentication: <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>)

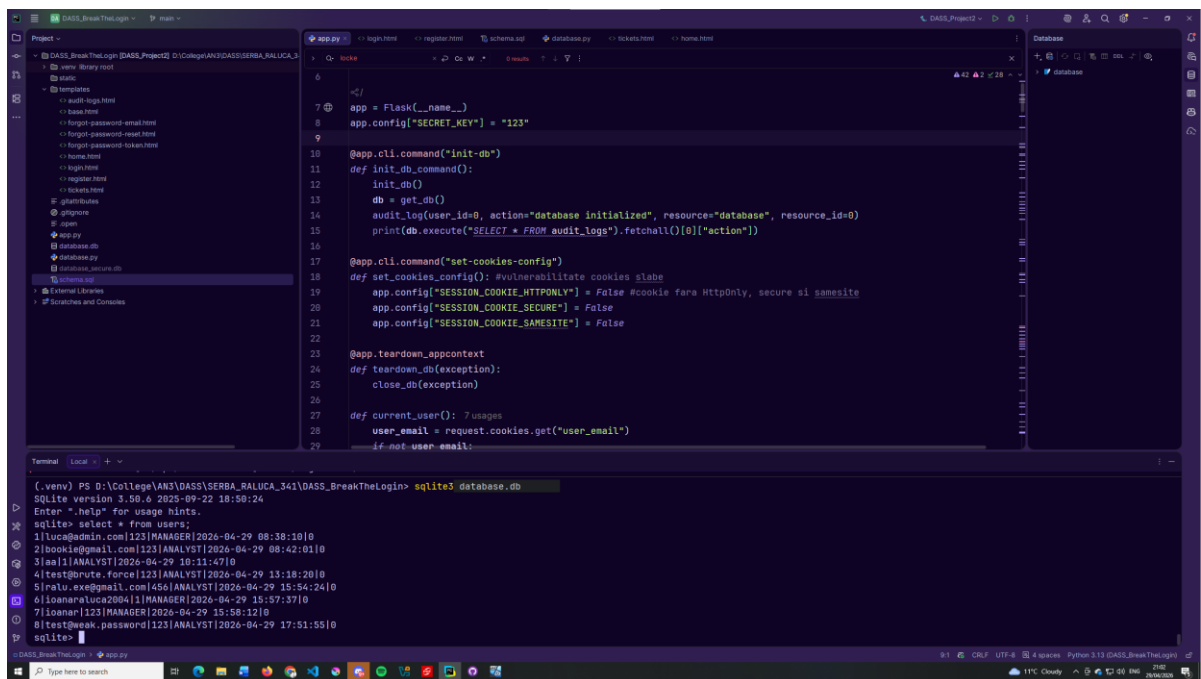
V. Demonstrare atac (PoC)

1. Password Policy Slab

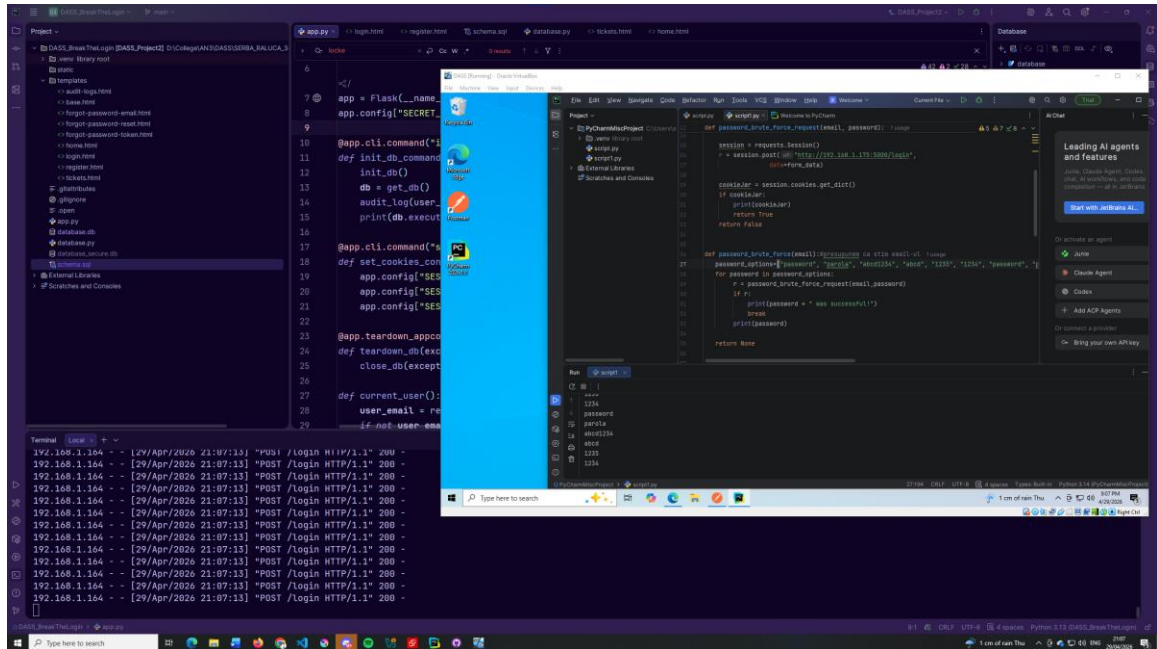
Creare cont cu parolă extrem de slabă (123) – primire confirmare creare cont.



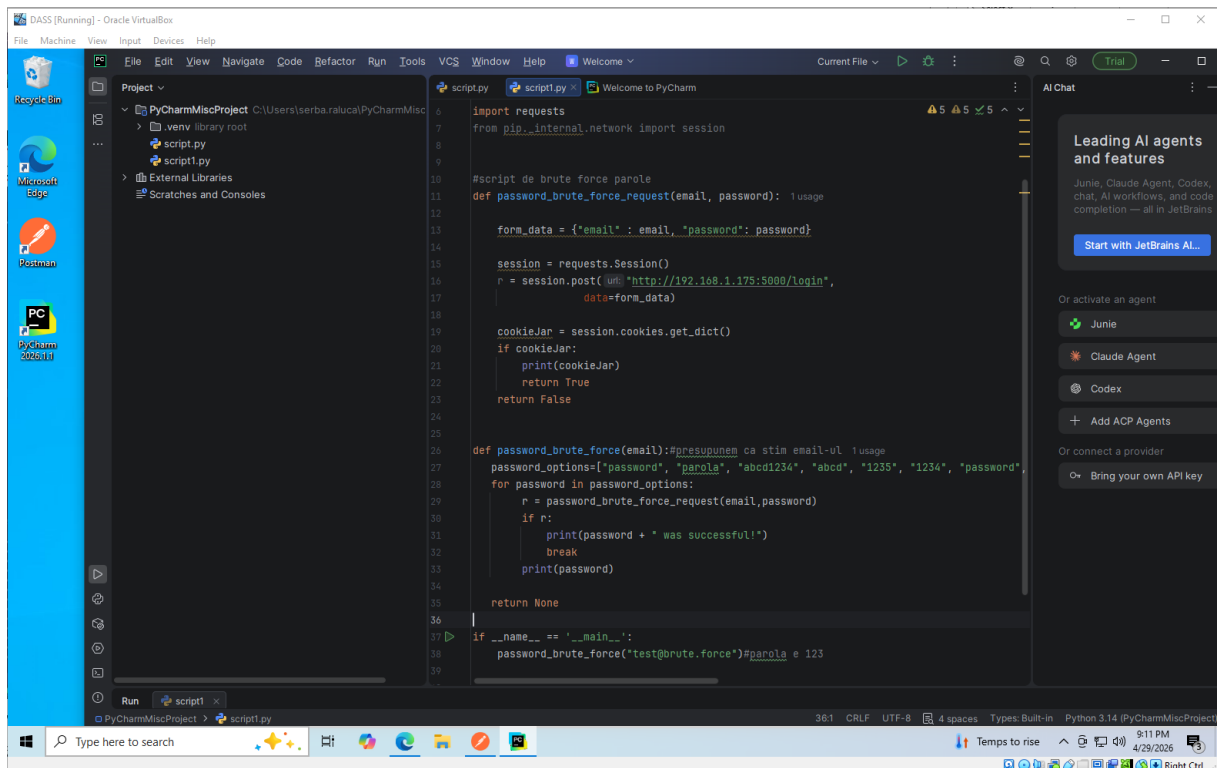
2. Stocare nesigură a parolelor – (Acces la DB simulat) – parole stocate în clar.



3. Brute force / lipsă rate limiting – Script care încearca un numar mare de parole în succesiune rapidă

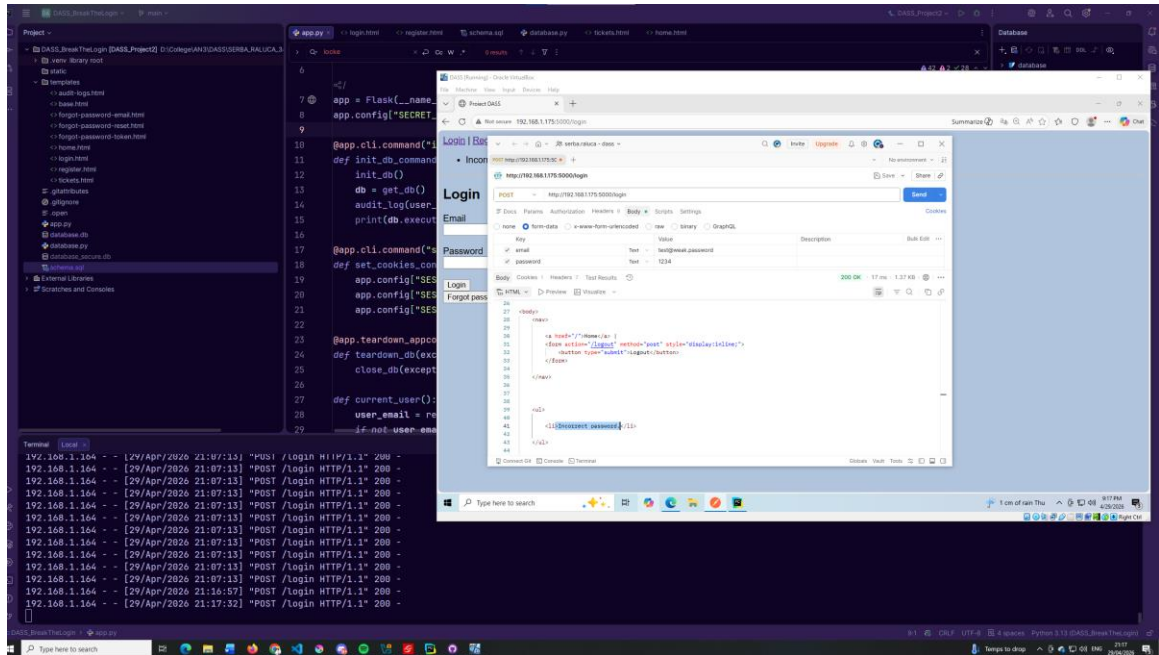


Se poate observa în terminalul host-ului și timestamp; scriptul este unul foarte simplu care doar trimite niste request-uri cu diferite parole către POST /login.

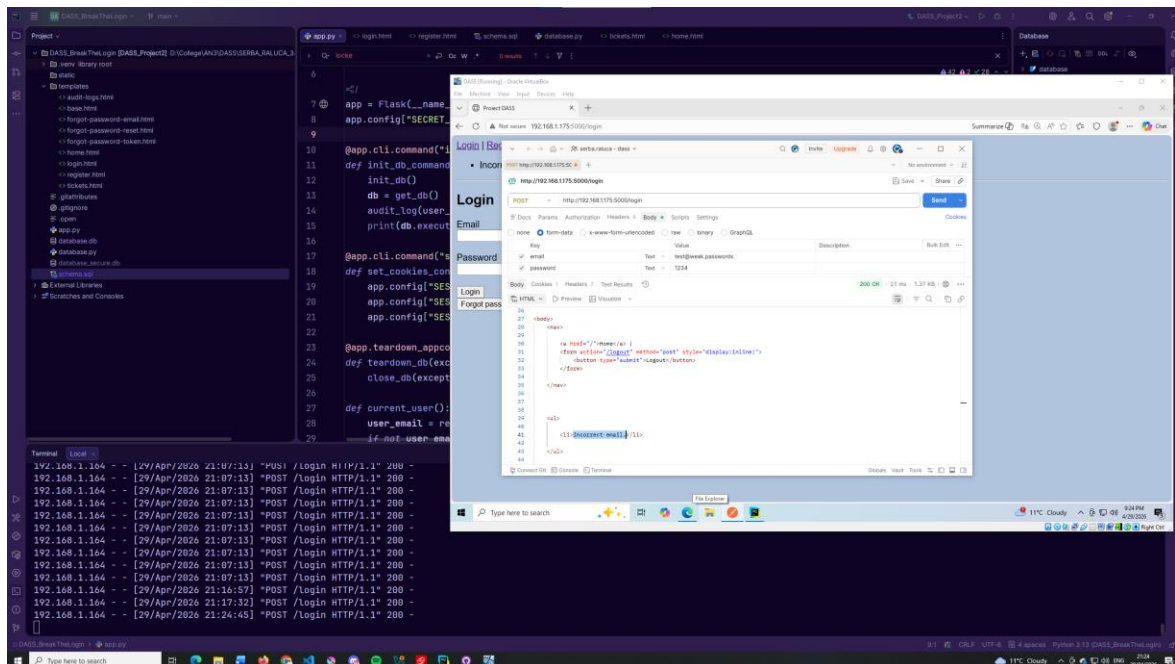


4. User Enumeration

La login cu un email pe care îl cunoaștem, dar o parolă gresită – primim avertismentul “Incorrect password.”

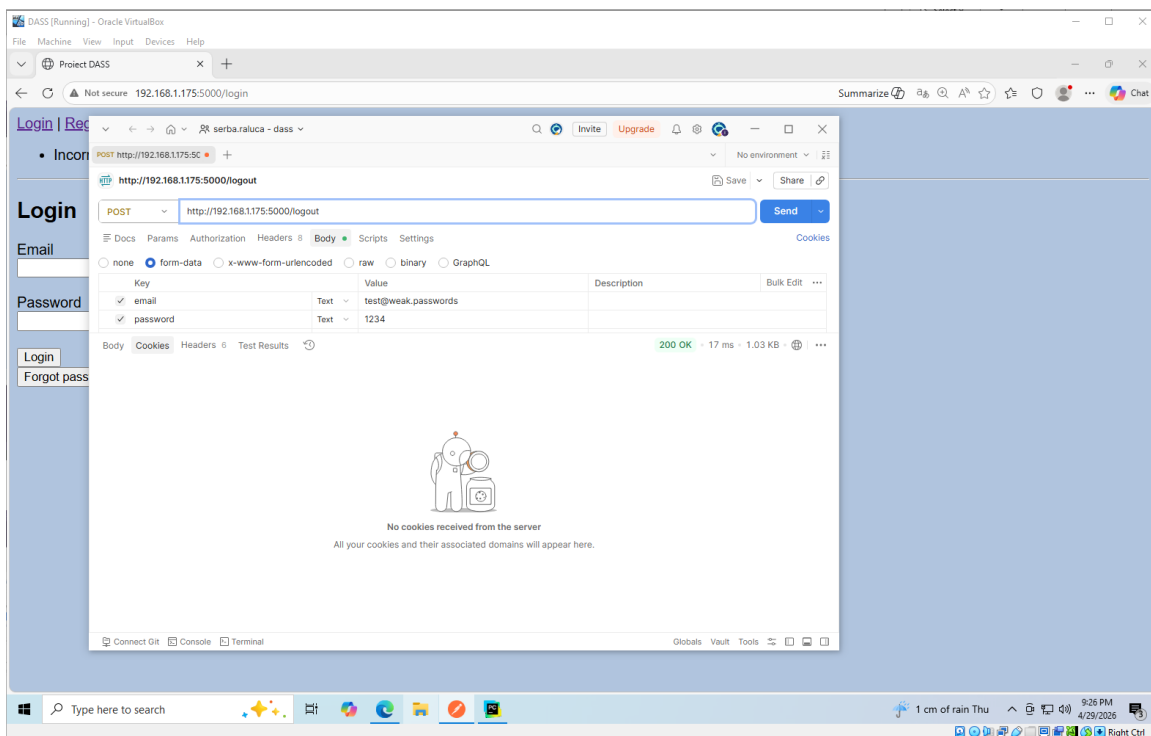


La login cu email incorect primim avertismentul “Incorrect email.”

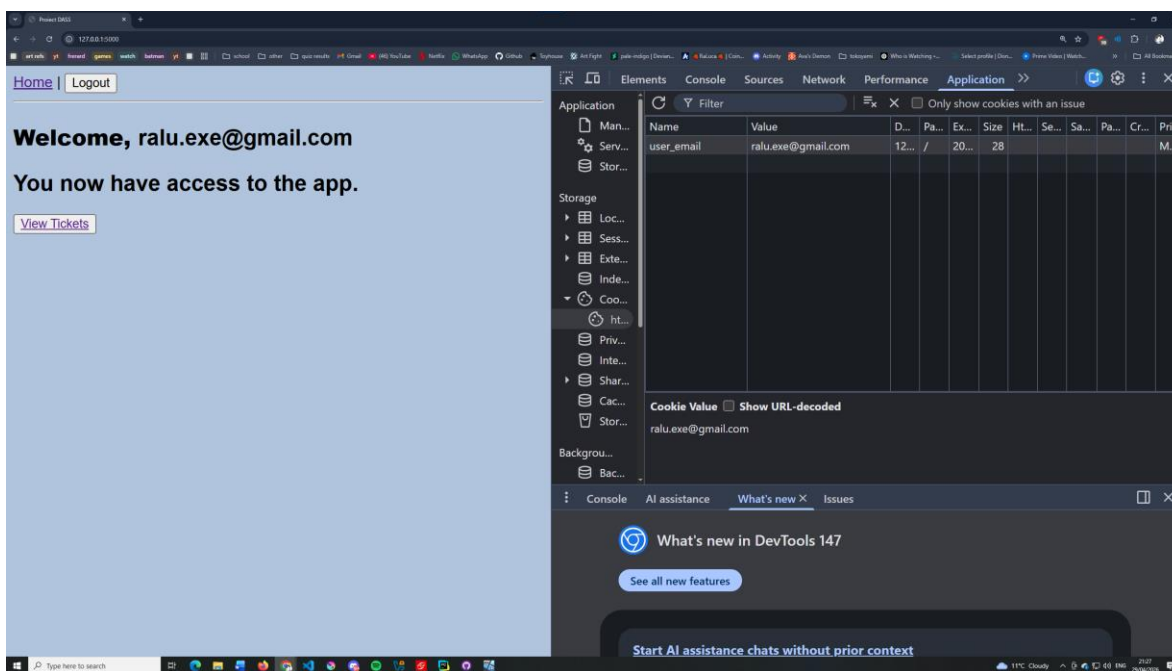


5. Gestionare nesigură a sesiunilor - Reutilizarea unui cookie

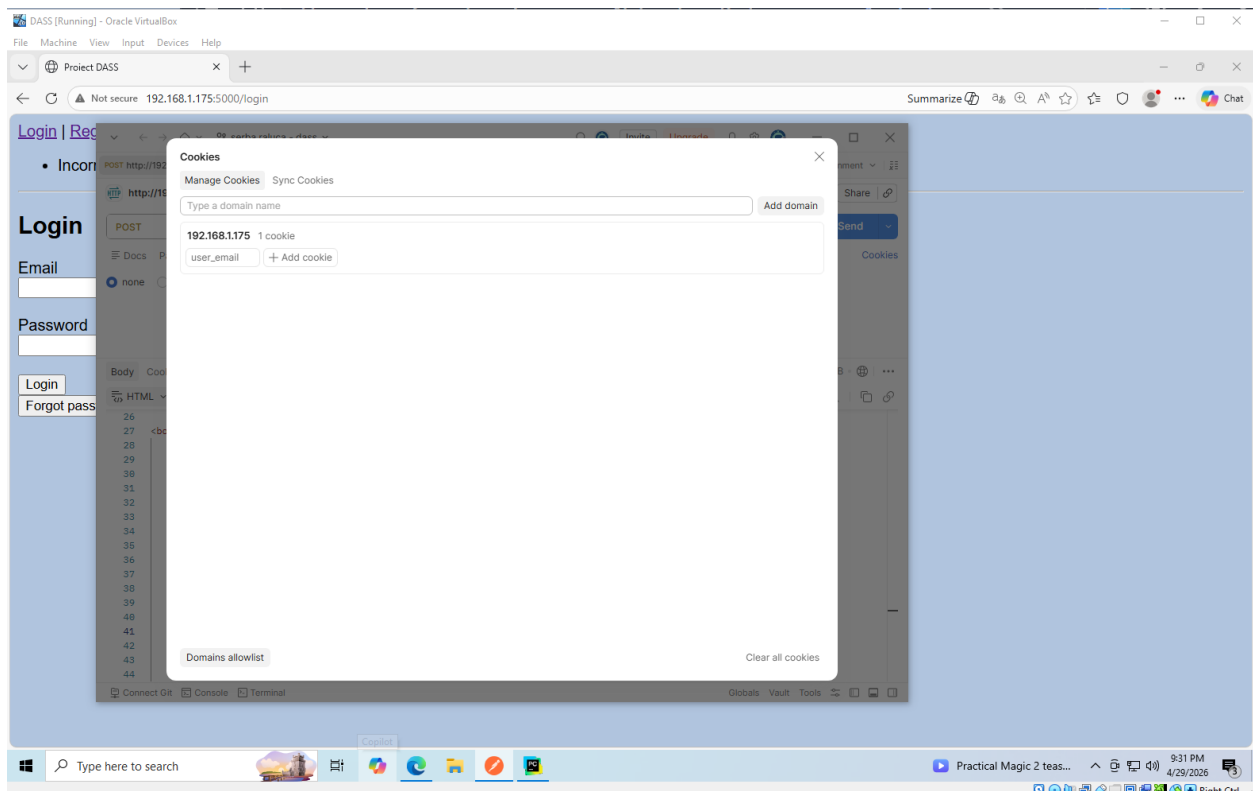
În VM: Nu avem cookies, nu suntem logați.



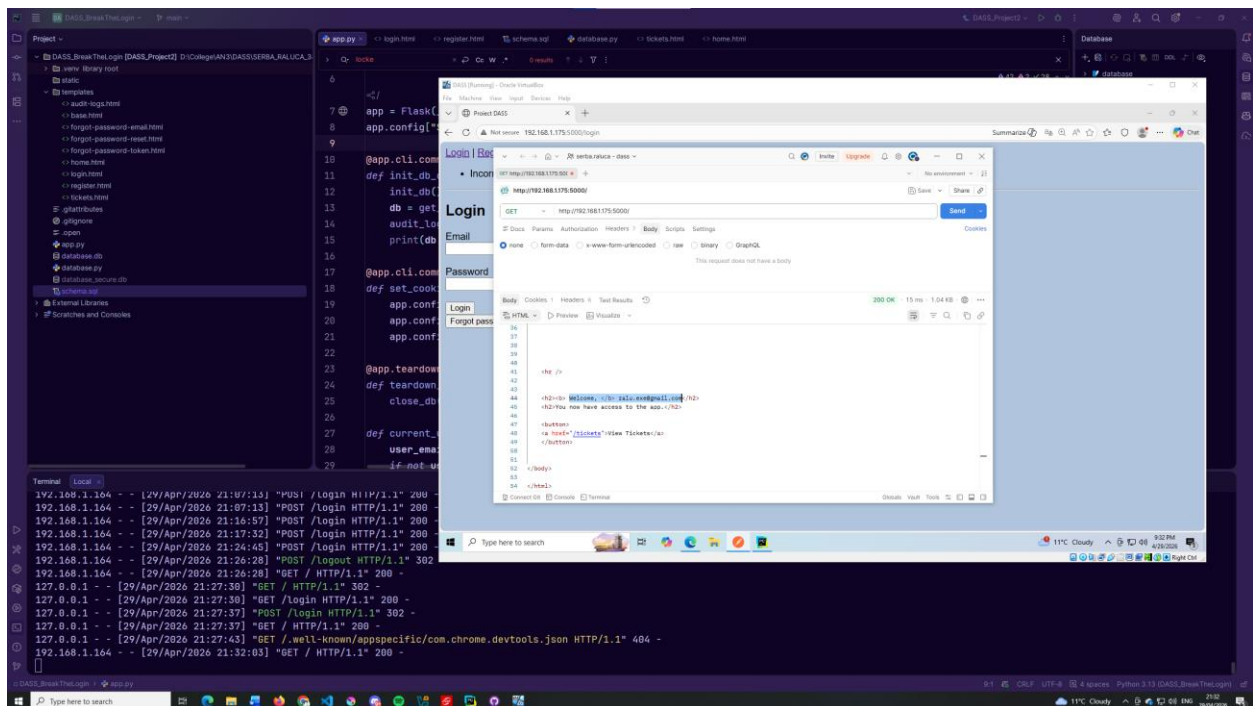
În Host: Ne-am autentificat cu success, se observa cookie-ul: `user_email=ralu.exe@gmail.com`



În VM, în Postman, simulăm ‚furarea’ cookie-ului.



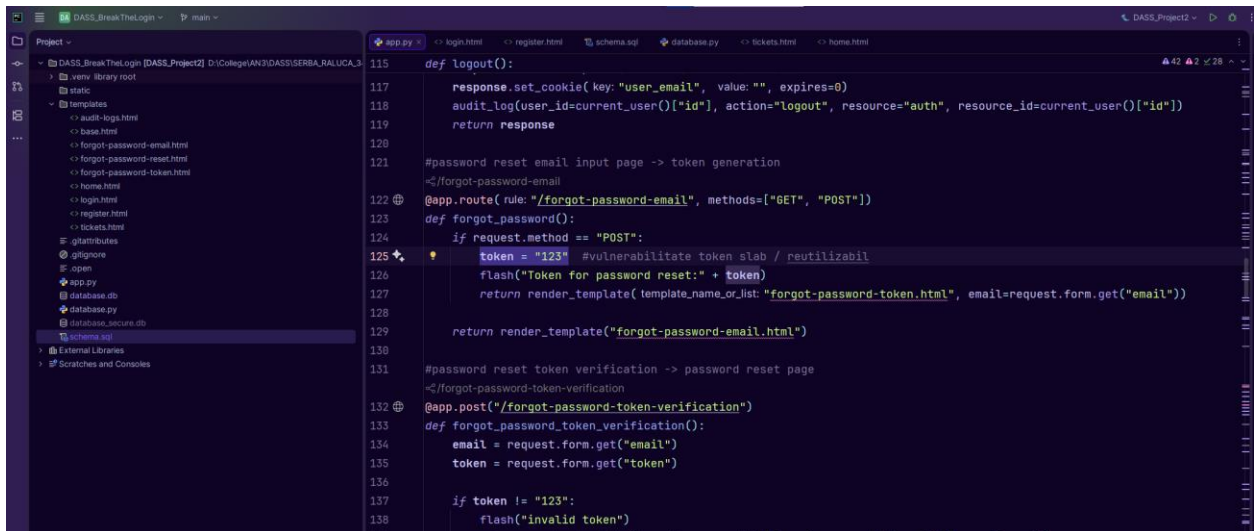
Și, fără să ne fi logat, facem o cerere către home, care ne tratează de parcă am fi înregistrați.



În audit log, se observă că am dat login pe o adresa ip (host) și logout de pe altă adresă (VM).

76	5	ralu.exe@gmail.com	login	auth	5	2026-04-29 18:27:36	127.0.0.1
77	5	ralu.exe@gmail.com	logout	auth	5	2026-04-29 18:34:25	192.168.1.164

6. Resetare parolă nesigură – Token reutilizabil, constant



```
115 def logout():
117     response.set_cookie(key="user_email", value="", expires=0)
118     audit_log(user_id=current_user()["id"], action="logout", resource="auth", resource_id=current_user()["id"])
119     return response
120
121 #password reset email input page -> token generation
122 ~</forgot-password-email
123
124 @app.route(rule="/forgot-password-email", methods=["GET", "POST"])
125 def forgot_password():
126     if request.method == "POST":
127         token = "123" #vulnerabilitate token slab / reutilizabil
128         flash("Token for password reset:" + token)
129         return render_template(template_name_or_list="forgot-password-token.html", email=request.form.get("email"))
130
131     return render_template("forgot-password-email.html")
132
133 #password reset token verification -> password reset page
134 ~</forgot-password-token-verification
135
136 @app.post("/forgot-password-token-verification")
137 def forgot_password_token_verification():
138     email = request.form.get("email")
139     token = request.form.get("token")
140
141     if token != "123":
142         flash("invalid token")
```

VI. Analiză impact

1. Password Policy Slab

Din cauza faptului că utilizatorii își pot crea conturi cu parolă foarte slabe, atacatorul are oportunitatea să încerce cele mai comune și simple parole sau să facă brute force fără un script complex; se ridică și riscul unui posibil atac de credential stuffing (în care atacatorul folosește date furate de pe alte site-uri pentru a sparge contul).

2. Stocare nesigură a parolelor

Dacă un atacator ar face rost de acces la baza de date, un simplu select i-ar oferi toate parolele tuturor conturilor.

3. Brute force / lipsă rate limiting

Atacatorul are oportunitatea și libertatea completă să încerce orice număr și combinație de parole printr-un atac de tip brut force.

4. User Enumeration

Prin precizarea la o încercare de login care dintre credentiale este greșită, oferim informație în plus atacatorului; Poate afla care conturi chiar există cu siguranță.

5. Gestionare nesigură a sesiunilor

Atacatorul poate avea foarte ușor access la cookie uri; acestea țin 24 de ore, și odată obținute le poate folosi pentru a se perfa autentificat în contul respective și are astfel acces la cont.

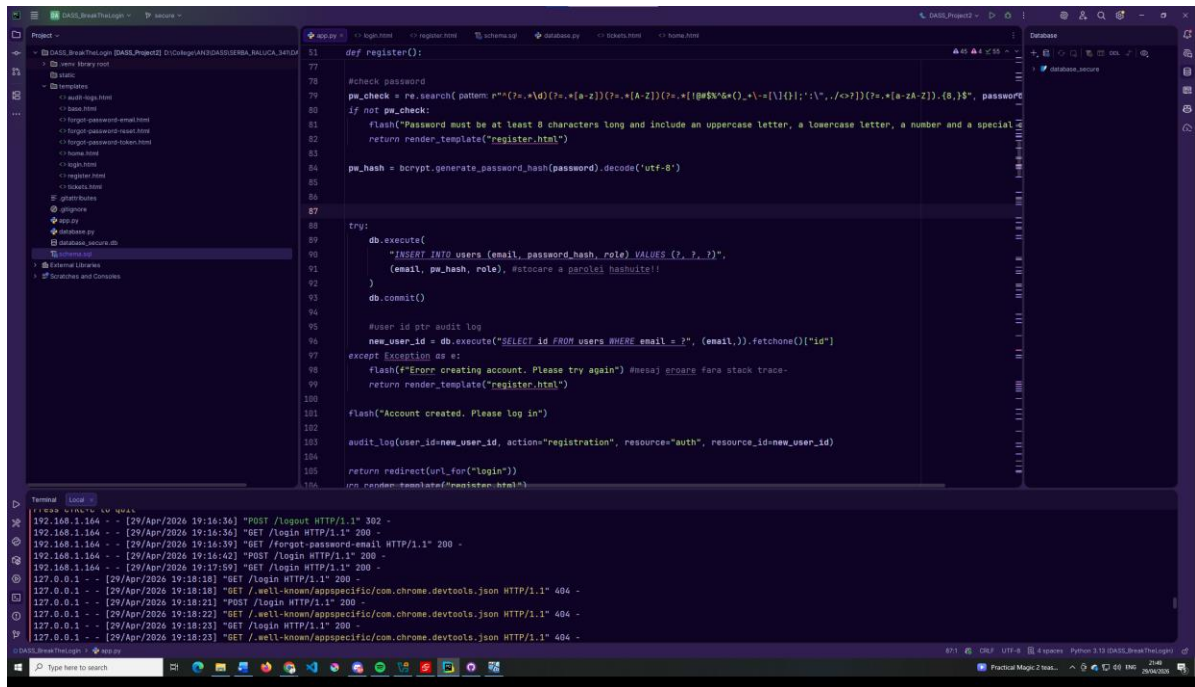
6. Resetare parolă nesigură

Atacatorul poate foarte ușor să ghicească / reutilizeze token-ul, astfel nu numai căpătând acces la cont prin schimbarea parolei fără drepturi, dar și blocând utilizatorul original din a se conecta.

VII. Implementare fix

1. Password Policy Slab

Implementare verificării asupra inputului de parolă că acesta să conțină măcar un uppercase, un lowercase, un caracter special și un număr. Am făcut această modificare și la register, și la resetarea parolei.



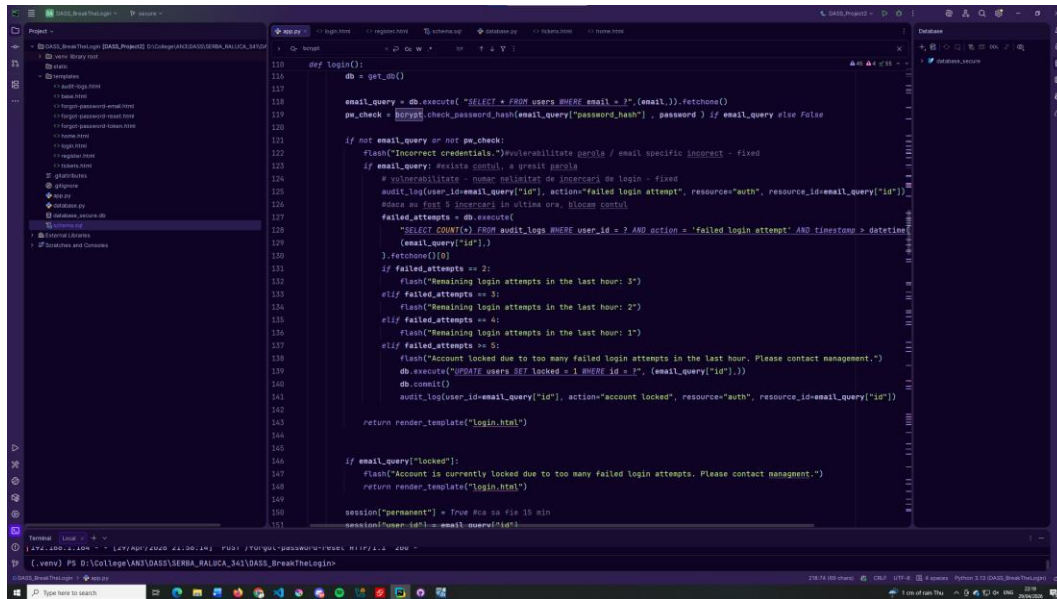
```
def register():
    77
    78     #check password
    79     pw_check = re.search(pattern=r"(?=[a-z])(?=[A-Z])(?=.*[@#$%^&*()_!~`{}|'\";:~.,/?])(?=.*[0-9])(?=[a-zA-Z]).{8,}$", password)
    80     if not pw_check:
    81         flash("Password must be at least 8 characters long and include an uppercase letter, a lowercase letter, a number and a special
    82         return render_template("register.html")
    83
    84     pw_hash = bcrypt.generate_password_hash(password).decode('utf-8')
    85
    86
    87     try:
    88         db.execute(
    89             "INSERT INTO users (email, password_hash, role) VALUES (?, ?, ?)",
    90             (email, pw_hash, role), #stocare a parolei hashuite!
    91         )
    92         db.commit()
    93
    94         #user id ptr audit log
    95         new_user_id = db.execute("SELECT id FROM users WHERE email = ?", (email,)).fetchone()[0]
    96
    97     except Exception as e:
    98         flash("Error creating account. Please try again!") #mesaj error fara stack trace-
    99         return render_template("register.html")
    100
    101     flash("Account created. Please log in")
    102
    103     audit_log(user_id=new_user_id, action="registration", resource="auth", resource_id=new_user_id)
    104
    105     return redirect(url_for("login"))
    106
    107     return render_template("register.html")
```

Terminal

```
192.168.1.164 - - [29/Apr/2026 19:16:36] "POST /logout HTTP/1.1" 302 -
192.168.1.164 - - [29/Apr/2026 19:16:36] "GET /login HTTP/1.1" 200 -
192.168.1.164 - - [29/Apr/2026 19:16:38] "GET /forgot-password-email HTTP/1.1" 200 -
192.168.1.164 - - [29/Apr/2026 19:16:42] "POST /login HTTP/1.1" 200 -
192.168.1.164 - - [29/Apr/2026 19:17:59] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:18:18] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:18:18] "GET /well-known/appspecific/com.chrome.devtools.json HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 19:18:21] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:18:22] "GET /.well-known/appspecific/com.chrome.devtools.json HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 19:18:23] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:18:23] "GET /well-known/appspecific/com.chrome.devtools.json HTTP/1.1" 404 -
```

2. Stocare nesigură a parolelor

Înainte de salvarea parolei în baza de date, am incriptat-o cu bcrypt (se poate observă în figura anterioară). Am implementat modificările necesare la register, la schimbarea parolei, și la verificarea de login.



```
def login():
    db = get_db()

    email_query = db.execute("SELECT * FROM users WHERE email = ?", (email,)).fetchone()
    pw_check = bcrypt.check_password_hash(email_query["password_hash"], password) if email_query else False

    if not email_query or not pw_check:
        flash("Incorrect credentials." + "Vulnerabilitate parola / email specific incorrect - fixed")
        if email_query: revizita_contul, a gresit parola
        # vulnerabilitate - numar nelinit de incercari de login - fixed
        audit_log(user_id=email_query["id"], action="failed login attempt", resource="auth", resource_id=email_query["id"])
        # daca au fost 5 incercari in ultima ora, blocaz contul
        failed_attempts = db.execute(
            "SELECT COUNT(*) FROM audit_logs WHERE user_id = ? AND action = 'failed login attempt' AND timestamp > datetime(
                email_query["id"],
            ).fetchone()[0]
        )
        if failed_attempts == 2:
            flash("Remaining login attempts in the last hour: 3")
        elif failed_attempts == 3:
            flash("Remaining login attempts in the last hour: 2")
        elif failed_attempts == 4:
            flash("Remaining login attempts in the last hour: 1")
        elif failed_attempts == 5:
            flash("Account locked due to too many failed login attempts in the last hour. Please contact management.")
            db.execute("UPDATE users SET locked = 1 WHERE id = ?", (email_query["id"],))
            db.commit()
            audit_log(user_id=email_query["id"], action="account locked", resource="auth", resource_id=email_query["id"])

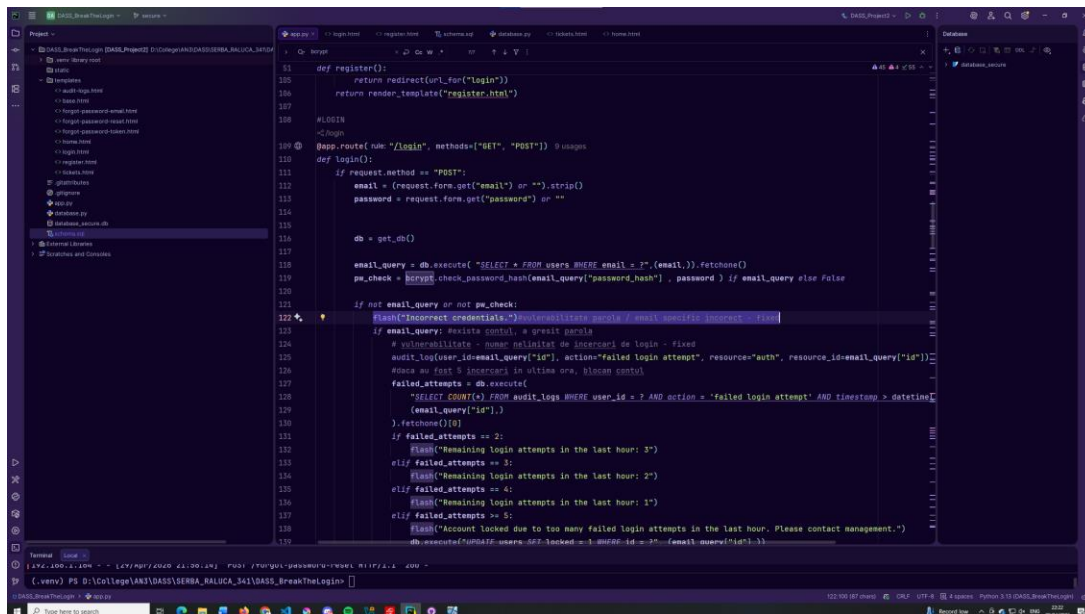
    return render_template("login.html")

if email_query["locked"]:
    flash("Account is currently locked due to too many failed login attempts. Please contact management.")
    return render_template("login.html")

session["permanent"] = True # ca sa fie 15 min
session["user_id"] = email_query["id"]
```

4. User Enumeration

La o logare eşuată, indiferent de credentialele greşite se afişează un mesaj unic.



```
def register():
    return redirect(url_for("login"))
    return render_template("register.html")

@login_required
def login():
    if request.method == "POST":
        email = (request.form.get("email") or "").strip()
        password = request.form.get("password") or ""

        db = get_db()

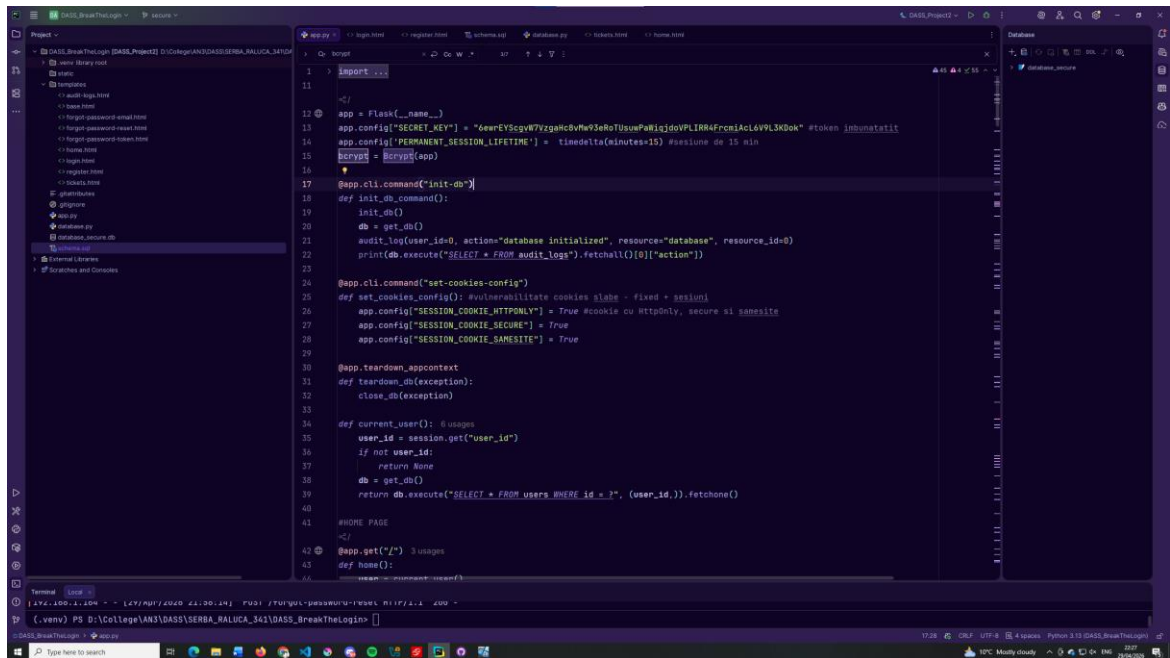
        email_query = db.execute("SELECT * FROM users WHERE email = ?", (email,)).fetchone()
        pw_check = bcrypt.check_password_hash(email_query["password_hash"], password) if email_query else False

        if not email_query or not pw_check:
            flash("Incorrect credentials." + "Vulnerabilitate parola / email specific incorrect - fixed")
            if email_query: revizita_contul, a gresit parola
            # vulnerabilitate - numar nelinit de incercari de login - fixed
            audit_log(user_id=email_query["id"], action="failed login attempt", resource="auth", resource_id=email_query["id"])
            # daca au fost 5 incercari in ultima ora, blocaz contul
            failed_attempts = db.execute(
                "SELECT COUNT(*) FROM audit_logs WHERE user_id = ? AND action = 'failed login attempt' AND timestamp > datetime(
                    email_query["id"],
                ).fetchone()[0]
            )
            if failed_attempts == 2:
                flash("Remaining login attempts in the last hour: 3")
            elif failed_attempts == 3:
                flash("Remaining login attempts in the last hour: 2")
            elif failed_attempts == 4:
                flash("Remaining login attempts in the last hour: 1")
            elif failed_attempts == 5:
                flash("Account locked due to too many failed login attempts in the last hour. Please contact management.")
                db.execute("UPDATE users SET locked = 1 WHERE id = ?", (email_query["id"],))
                db.commit()
```

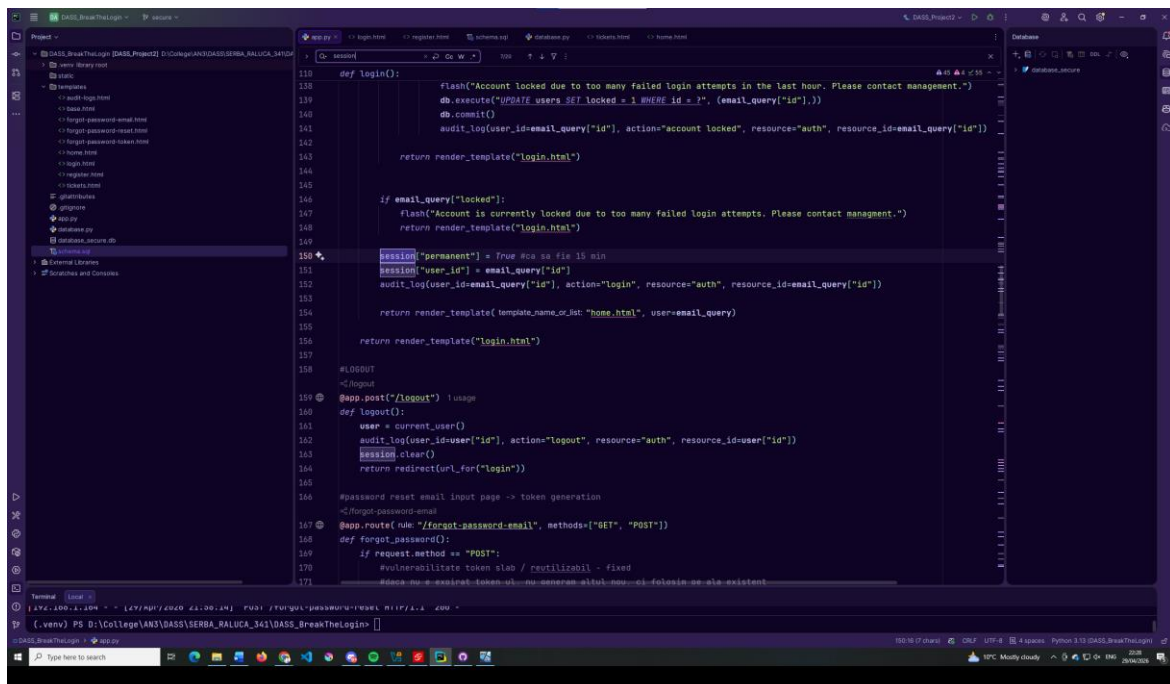
5. Gestionare nesigură a sesiunilor

În loc de cookie-uri obișnuite, am trecut la a utiliza sesiuni Flask, care sunt cookie-uri mai criptate, mai sigure și puțin mai ușor de utilizat. Am îmbunătățit parolă aplicație, care

încrîptează sesiunile (era "123" înainte). M-am asigurat că cookie-urile au HttpOnly, SameSite și Secure setate la True, pentru orice alt cookie folosit în aplicație, și am modificat sesiunile, care în mod normal țin 31 de zile, să aibă un ciclu de viață de doar 15 minute.



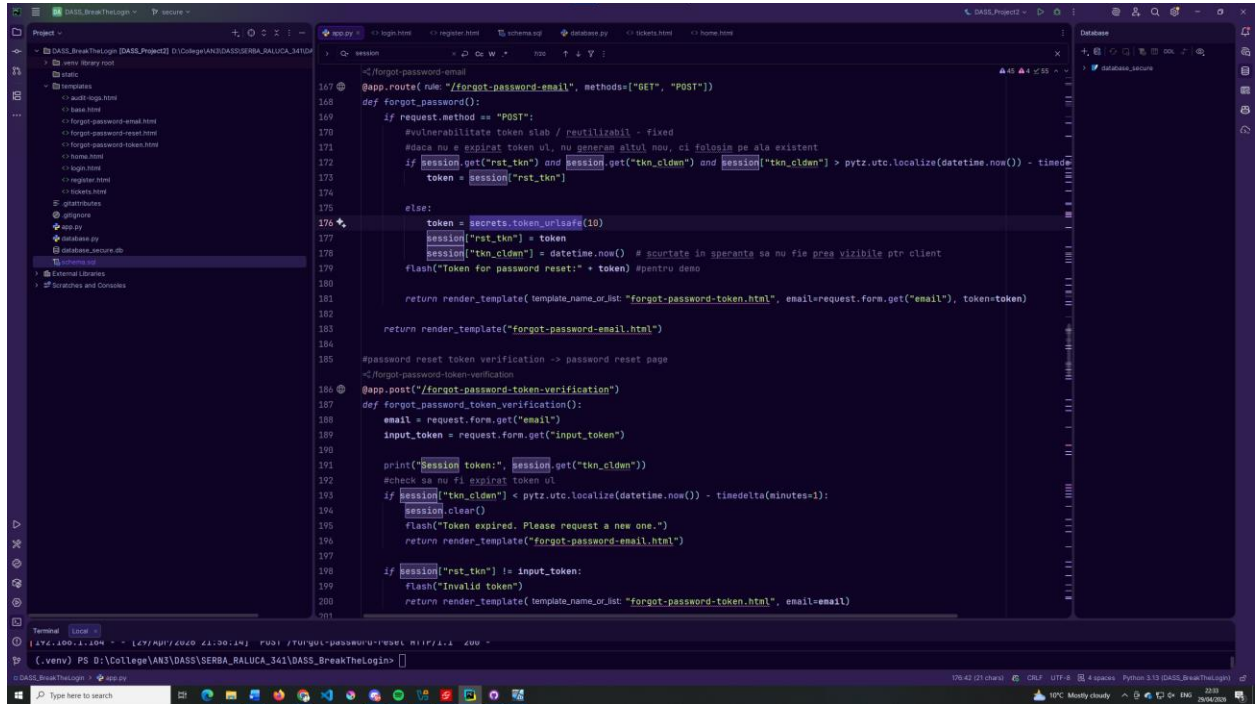
```
1 import sys
2
3
4
5
6
7
8
9
10
11
12 app = Flask(__name__)
13 app.config["SECRET_KEY"] = "6ewEYsgv7Vzgmc6vhw3eR0tU5uePa8igjdvLIR84fcmAcL6V9LSK8ok" #token #inutilizat
14 app.config["PERMANENT_SESSION_LIFETIME"] = timedelta(minutes=15) #sesiune de 15 min
15 @contextmanager
16 def _acquire_ctx():
17     pass
18
19 def init_db():
20     init_db()
21     db = get_db()
22     audit_log(user_id=0, action="database initialized", resource="database", resource_id=0)
23     print(db.execute("SELECT * FROM audit_log").fetchall())
24
25 @app.cli.command("init-db")
26 def init_db():
27     init_db()
28
29 def set_cookies_config():
30     #vulnerabilitate cookies alabe - fixed + session
31     app.config["SESSION_COOKIE_HTTPONLY"] = True #cookie cu HttpOnly, secure si samesite
32     app.config["SESSION_COOKIE_SECURE"] = True
33     app.config["SESSION_COOKIE_SAMESITE"] = True
34
35 @app.teardown_appcontext
36 def teardown_db(exception):
37     close_db(exception)
38
39 def current_user():
40     user_id = session.get("user_id")
41     if not user_id:
42         return None
43     db = get_db()
44     return db.execute("SELECT * FROM users WHERE id = ?", (user_id,)).fetchone()
45
46 #HOME PAGE
47
48 @app.get("/")
49 def home():
50     return render_template("home.html")
```



```
110 def login():
111     flash("Account locked due to too many failed login attempts in the last hour. Please contact management.")
112     db.execute("UPDATE users SET locked = 1 WHERE id = ?", (email_query["id"],))
113     db.commit()
114     audit_log(user_id=email_query["id"], action="account locked", resource="auth", resource_id=email_query["id"])
115     return render_template("login.html")
116
117 if email_query["locked"]:
118     flash("Account is currently locked due to too many failed login attempts. Please contact management.")
119     return render_template("login.html")
120
121 session["permanent"] = True #do so for 15 min
122 session["user_id"] = email_query["id"]
123 audit_log(user_id=email_query["id"], action="login", resource="auth", resource_id=email_query["id"])
124 return render_template(template_name_or_list="home.html", user=email_query)
125
126 return render_template("login.html")
127
128 #Logout
129 @app.post("/logout")
130 def logout():
131     user = current_user()
132     audit_log(user_id=user["id"], action="logout", resource="auth", resource_id=user["id"])
133     session.clear()
134     return redirect(url_for("login"))
135
136 #password reset email input page -> token generation
137 @app.route("/forgot-password-email", methods=["GET", "POST"])
138 def forgot_password_email():
139     if request.method == "POST":
140         #vulnerabilitate token slab / reutilizabil - fixed
141         #do not re-exist token or no generation of token - do not exist
142         #do not re-exist token or no generation of token - do not exist
```


6. Resetare parolă nesigură

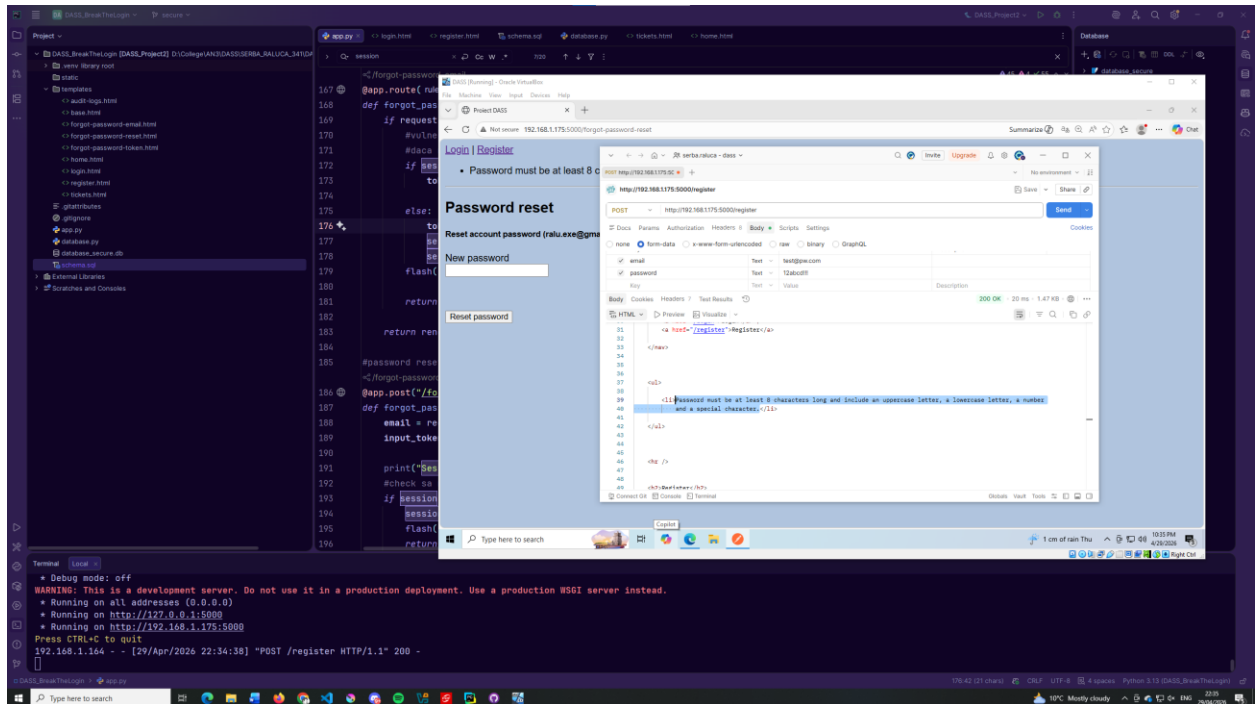
În loc de token static, am generat tokene unice cu ajutorul `secrets.token_urlsafe`, pe care îl rețin în sesiune, împreună cu timestamp-ul generării acestuia pentru a îi impune un ciclu de viață de 15 minute. Dacă nu trecut 15 minute, token-ul rămâne valid, însă dacă trec procesul nu poate fi continuat și trebuie solicitat un token nou. Aceste variabile fiind în sesiune, rămân criptate.



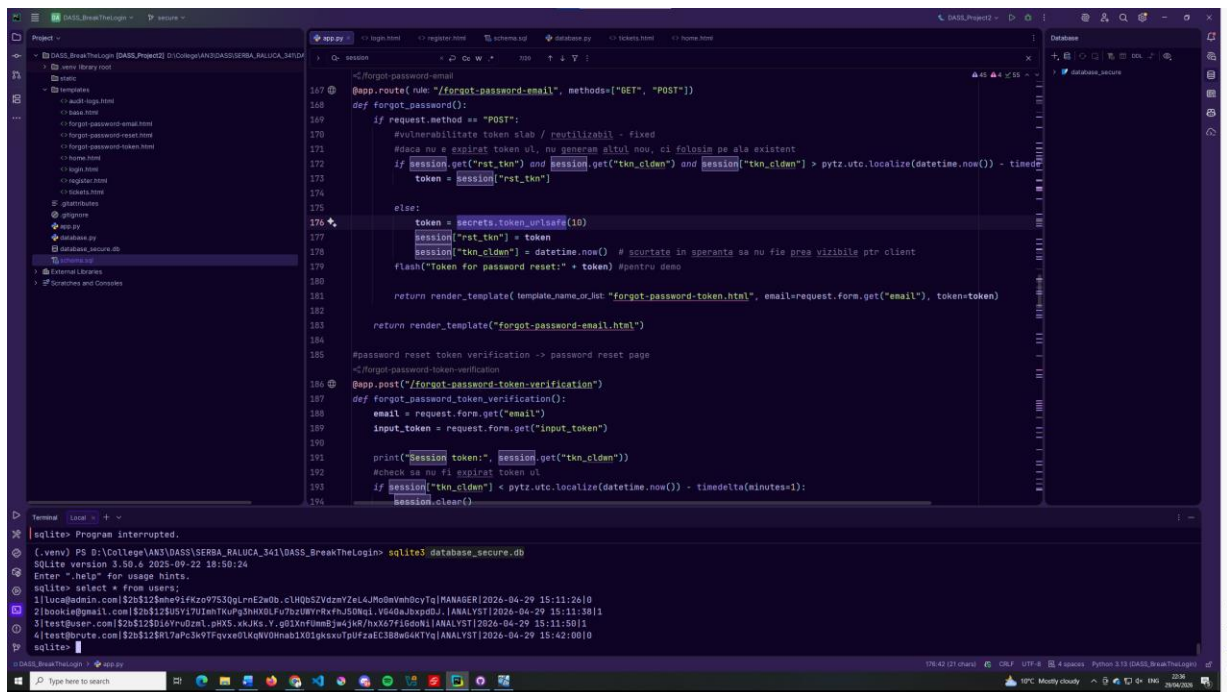
```
167 @app.route("/forgot-password-email", methods=["GET", "POST"])
168 def forgot_password():
169     if request.method == "POST":
170         #vulnerabilitate token slab / reutilizabil - fixed
171         #daca nu e expirat token ul, nu generam altul nou, ci folosim pe ala existent
172         if session.get("rst_tkn") and session.get("tkn_cldm") and session["tkn_cldm"] > pytz.utc.localize(datetime.now()) - timedelta(minutes=15):
173             token = session["rst_tkn"]
174         else:
175             token = secrets.token_urlsafe(10)
176             session["rst_tkn"] = token
177             session["tkn_cldm"] = datetime.now() # scrie in speranta sa nu fie prea vizibil ptr client
178             flash("Token for password reset:" + token) #pentru demo
179             return render_template("forgot-password-token.html", email=request.form.get("email"), token=token)
180         return render_template("forgot-password-email.html")
181
182 #password reset token verification -> password reset page
183 @app.post("/forgot-password-token-verification")
184 def forgot_password_token_verification():
185     email = request.form.get("email")
186     input_token = request.form.get("input_token")
187
188     print("Session token:", session.get("tkn_cldm"))
189     #check sa nu fi expirat token ul
190     if session["tkn_cldm"] < pytz.utc.localize(datetime.now()) - timedelta(minutes=15):
191         session.clear()
192         flash("Token expired. Please request a new one.")
193         return render_template("forgot-password-email.html")
194     if session["rst_tkn"] != input_token:
195         flash("Invalid token")
196         return render_template(template_name_or_list="forgot-password-token.html", email=email)
```

VIII. Re-test

1. Password Policy Slab – Eroare la crearea unui cont cu parolă slabă

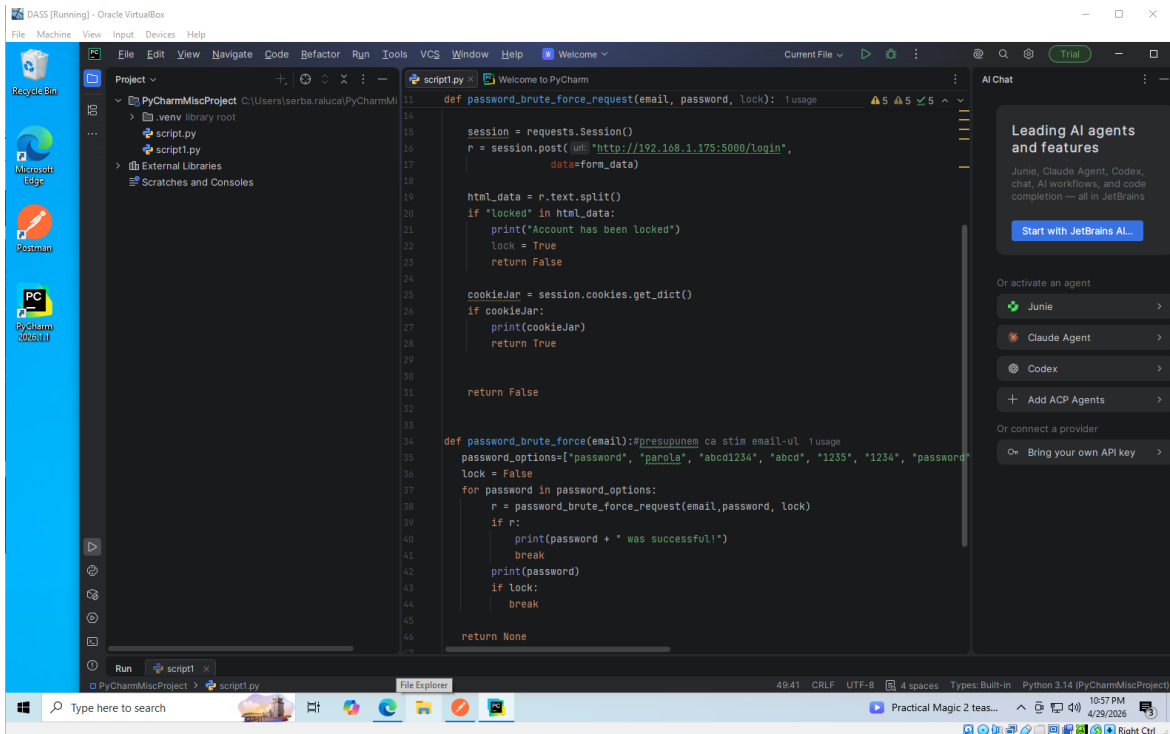
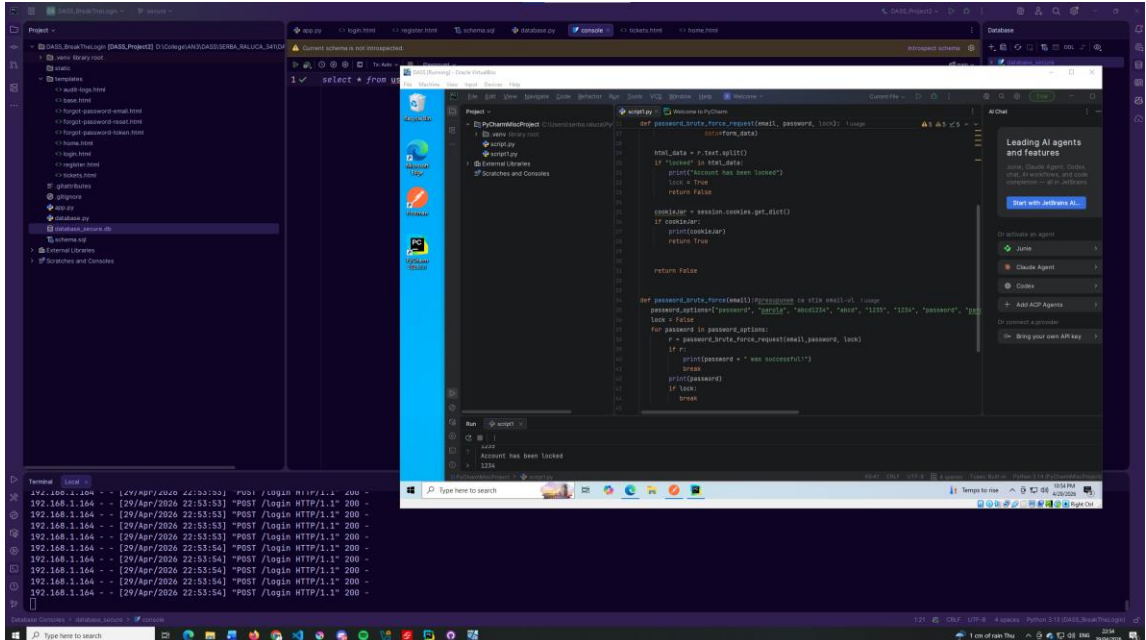


2. Stocare nesigură a parolelor – În baza de date, parolele sunt criptate.

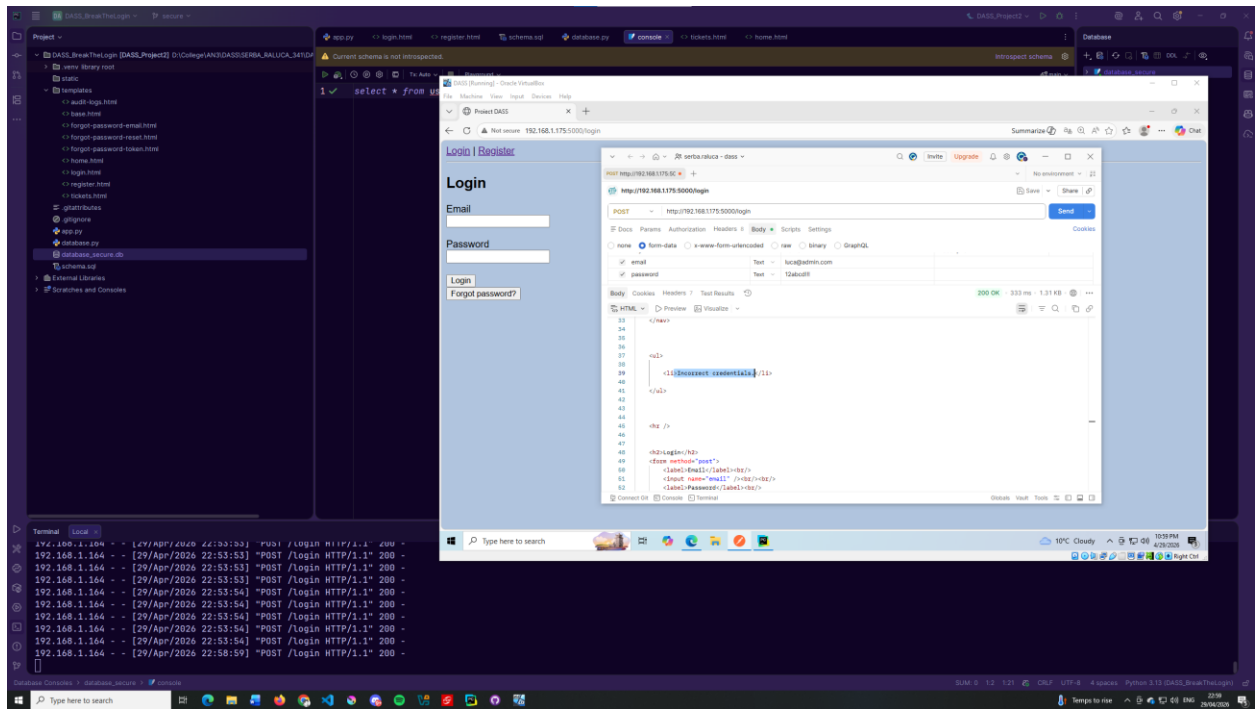


3. Brute force / lipsă rate limiting

Scriptul este foarte asemănător cu cel anterior, ușor modificat să extragă html-ul din răspuns și să caute dacă apare cuvântul ,locked' din avertisment.

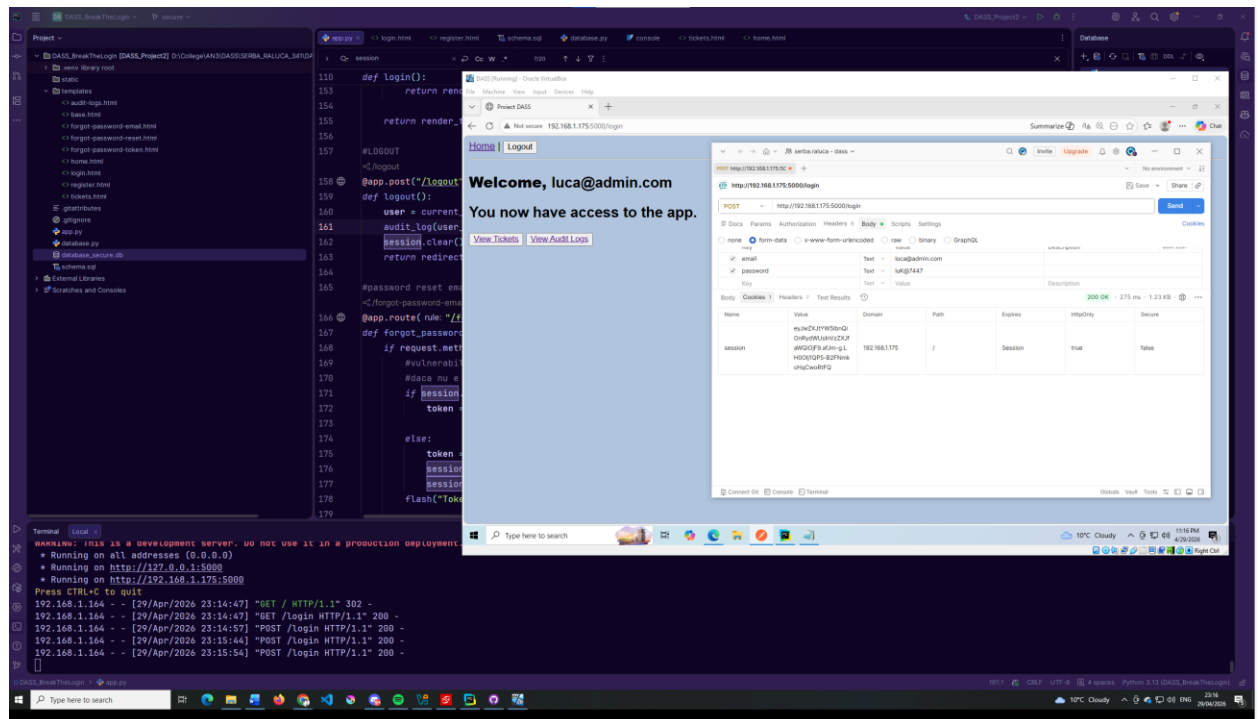


4. User Enumeration – Afisează eroarea nespecificată – “Incorrect credentials.”

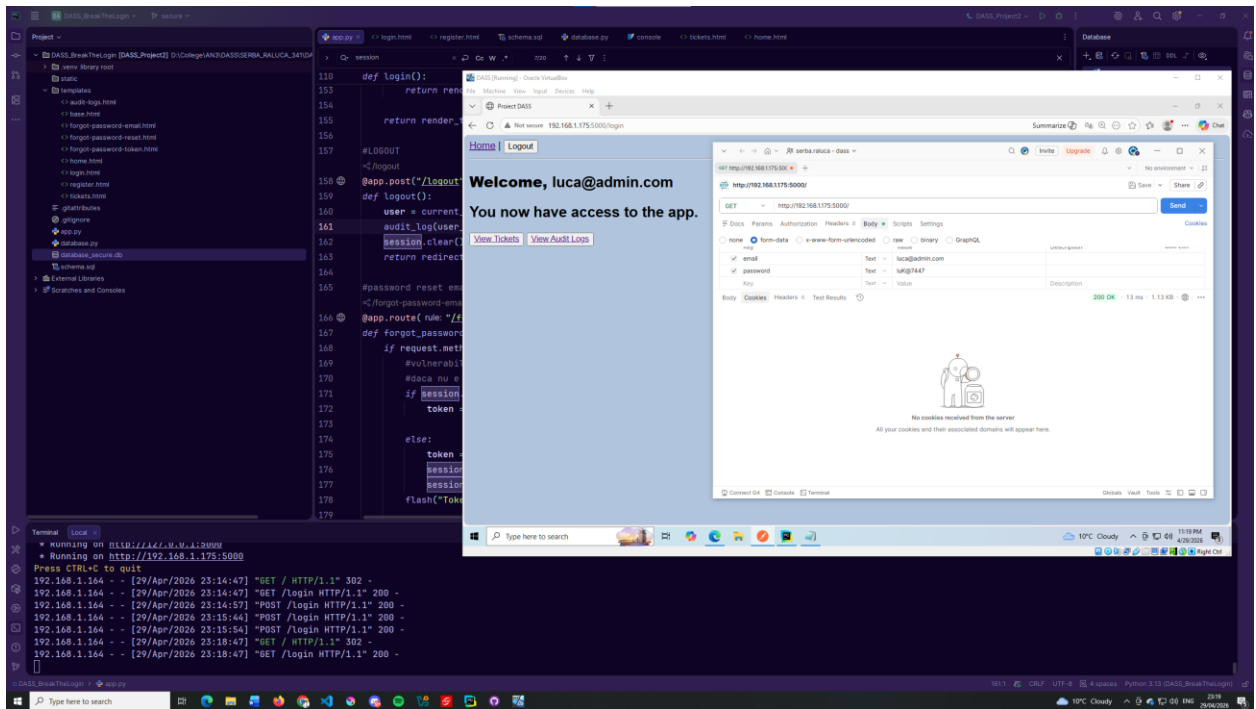


5. Gestionare nesigură a sesiunilor

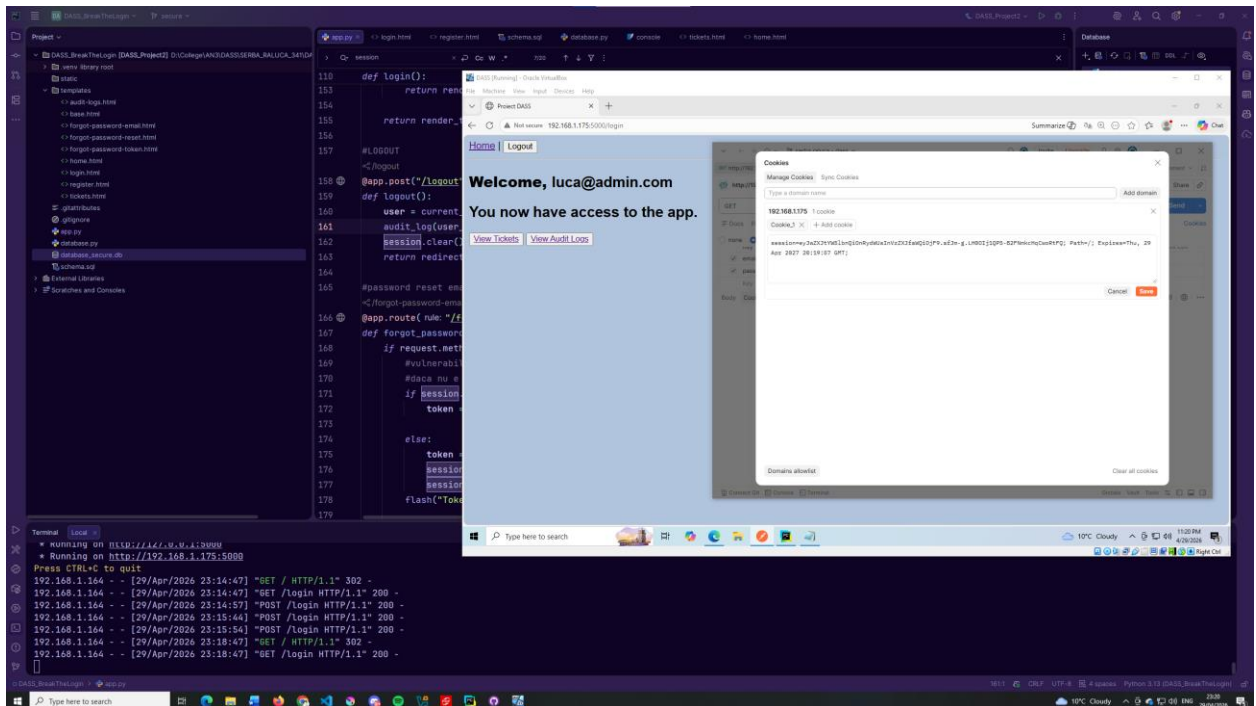
Login la 23:16 (ciclu de viață temporar setat la 1 minut pentru demo) – primire token sesiune.



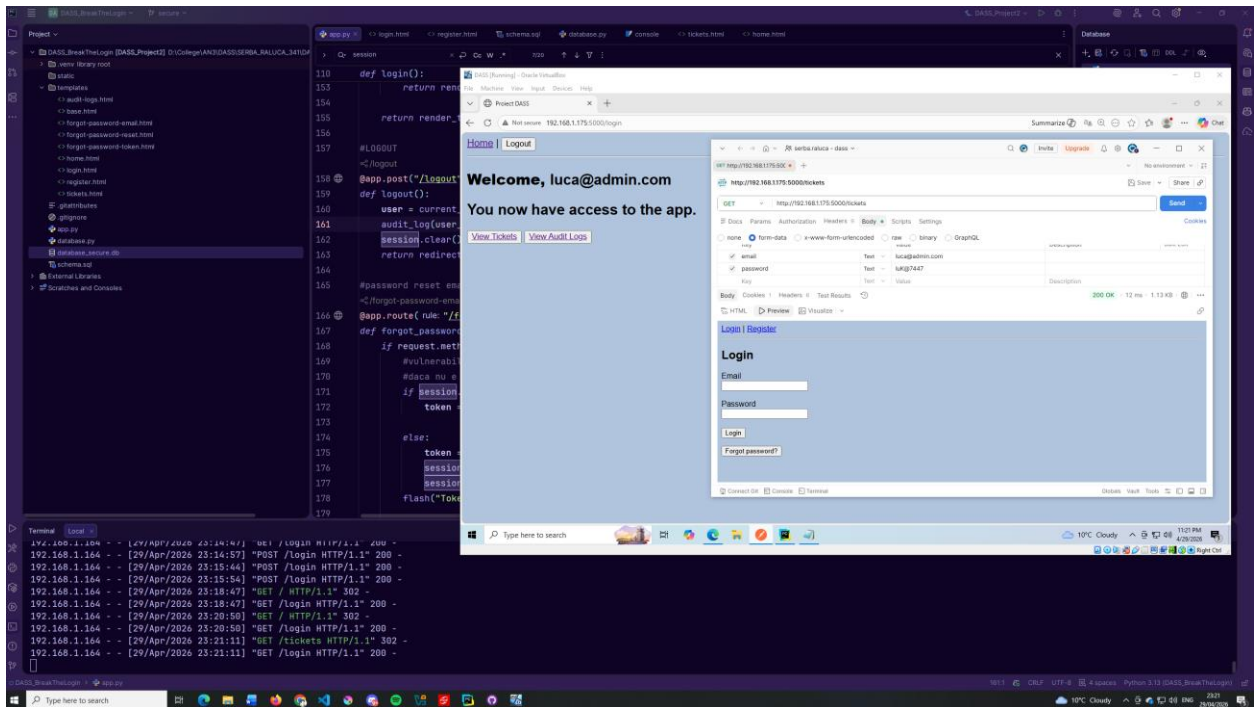
Rețin valoarea cookie-ului în clipboard. Fără să fi dat logout, peste câteva minute a expirat, nu mai este nici-un cookie setat



Îl trimit de mână, folosind valoarea reținută în clipboard.



Chiar dacă cookie-ul a fost trimis și setat, utilizatorul nu este considerat autentificat: la cererea de /home și /tickets este redirectionat la login.



6. Resetare parolă nesigură

Prin faptul că token-ul are o viață limitată și este mult mai complex și unic, atacul de brute-force își pierde mult probabilitatea să funcționeze.

IX. Audit & logging

Pentru a trasa acțiunile unui utilizator, în baza de date avem o tabela numită audit logs care înregistrează în tabel datele referitoare unei acțiuni, incluzând user_id, acțiune, resursă utilizată, id-ul acesteia, timestamp și adresa ip a utilizatorului. Se înregistrează date în tabel la îndeplinirea următoarelor acțiuni: initializarea bazei de date, înregistrarea unui nou cont, logarea cu succes a unui utilizator, logout, resetarea parolei unui cont, editarea unui ticket, crearea unui ticket, iar în varianta securizată se ține cont și de încercări eșuate de login și blocarea unui cont.

17	3	aa	registration	auth	3	2026-04-29 10:11:47	127.0.0.1
18	1	luca@admin.com	login	auth	1	2026-04-29 11:41:51	127.0.0.1
19	1	luca@admin.com	login	auth	1	2026-04-29 11:45:07	192.168.1.164
20	1	luca@admin.com	logout	auth	1	2026-04-29 11:45:35	192.168.1.164
21	1	luca@admin.com	logout	auth	1	2026-04-29 11:47:56	127.0.0.1
22	1	luca@admin.com	reset password	auth	1	2026-04-29 11:49:15	192.168.1.164
23	1	luca@admin.com	login	auth	1	2026-04-29 11:49:19	192.168.1.164
24	1	luca@admin.com	create ticket	ticket	2	2026-04-29 11:49:36	192.168.1.164
25	1	luca@admin.com	update ticket	ticket	1	2026-04-29 11:49:40	192.168.1.164

X. Concluzii

În urmă construirii unei aplicații intenționat vulnerabile și atacarea acesteia, am învățat să mă pun în poziția de atacator, să consider mai atent scopul anumitor acțiuni și informații și să înțeleg mai bine anumite bune-practici de codat pe care anterior le luăm ca atare. Unele aspecte care pot părea insignificative la prima vedere pot cu adevărat să facă diferența, important fiind să înțelegem cum funcționează. Perspectiva de a extrage informații din propria aplicație mi-a deschis ochii asupra faptului că și unele mici informații pot ajuta foarte mult un atacator, și trebuie găsit un echilibru între confortul și ajutorarea unui client, și descurajarea unui atacator. Această perspectivă chiar m-a făcut să regandesc anumite acțiuni la care, de obicei, nu aș fi stat pe gânduri, și să realizez potențialul risc pe care îl pun asupra utilizatorilor și aplicației create.

În final, consider acest proiect un exercițiu foarte bun, care pune accent pe niște practici foarte importante și bune de care orice programator ar trebui să țină cont.